

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Информатика и системы управления”
Кафедра “Системы обработки информации и управления”



Дисциплина “Парадигмы и конструкции языков программирования”

Отчет по домашнему заданию на тему
“Применение методологий Continuous integration и Continuous Delivery при
разработке ПО”

Выполнил:
Студент группы ИУ5-35Б
Иванов Б. М.
Преподаватель:
Нардид А. Н.

Москва 2025

Полученное изначально задание

1. Выберите язык программирования (который Вы ранее не изучали) и (1) напишите по нему реферат с примерами кода или (2) реализуйте на нем небольшой проект (с детальным текстовым описанием).
2. Реферат (проект) может быть посвящен отдельному аспекту (аспектам) языка или содержать решение какой-либо задачи на этом языке.
3. Необходимо установить на свой компьютер компилятор (интерпретатор, транспилятор) этого языка и произвольную среду разработки.
4. В случае написания реферата необходимо разработать и откомпилировать примеры кода (или модифицировать стандартные примеры).
5. В случае создания проекта необходимо детально комментировать код.
6. При написании реферата (создании проекта) необходимо изучить и корректно использовать особенности парадигмы языка и основных конструкций данного языка.
7. Приветствуется написание черновика статьи по результатам выполнения ДЗ. Черновик статьи может быть подготовлен группой студентов, которые исследовали один и тот же аспект в нескольких языках или решили одинаковую задачу на нескольких языках.

Выбранное и согласованное с преподавателем задание

1. Развернуть на локальном сервере GitLab.
2. Создать проект в развернутом GitLab`е.
3. Настроить GitLab-runner для проекта.
4. Написать gitlab-ci.yml.
5. Написать кодовую базу проекта.
6. Подключиться к созданному репозиторию и запустить туда код, пронаблюдать ход пайплайна и исправить выявленные проблемы.

Ход работы

Аннотация и технологический стек. Данный проект посвящен разворачиванию системы контроля версий GitLab в локальном окружении и последующей настройке автоматизированного пайплайна непрерывной интеграции и доставки (CI/CD). Проект выполнен с использованием Docker и Docker Compose для контейнеризации разворачиваемых сервисов. В качестве целевой платформы использована виртуальные машины с ОС Debian 13, развернутые на домашнем сервере OSDA IBM MSVSIA “Eclipse” виртуализации Альт-Виртуализация 11 (PROXMOX). Для настройки пайплайна и развертки с помощью Docker-compose GitLab применялся язык разметки YAML. Кодовая база проекта представляет собой простой веб сайт на HTML 5, выдающий статическую текстовую страницу.

Архитектура и принципы работы. Система состоит из следующих ключевых компонентов:

1. GitLab Server: Основной сервис, развернутый через официальный Docker-образ gitlab/gitlab-ee на отдельной виртуальной машине с помощью docker-compose для грамотного управления ресурсами. Сервер предоставляет веб-интерфейс для управления репозиториями, пользователями и пайплайнами.

Файл docker-compose.yml для GitLab

```
version: '3.8'
services:
  gitlab-db:
    image: postgres:15-alpine
    container_name: gitlab-postgres
    restart: always
    environment:
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: ${POSTGRES_DB}
      POSTGRES_INITDB_ARGS: "--encoding=UTF8 --lc-collate=C --lc-ctype=C"
      TZ: ${TZ}
    volumes:
      - ./data/postgres/data:/var/lib/postgresql/data
      - ./data/postgres/backups:/backups
    networks:
      - gitlab-network
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER}"]
      interval: 30s
      timeout: 10s
      retries: 5
    command:
      - "postgres"
      - "-c"
      - "max_connections=500"
      - "-c"
      - "shared_buffers=512MB"
      - "-c"
      - "effective_cache_size=2GB"
    ulimits:
```

```
nproc: 65535
nofile:
  soft: 65536
  hard: 65536
mem_limit: 4g
cpus: 2.0
gitlab-cache:
  image: redis:7-alpine
  container_name: gitlab-redis
  restart: always
  command:
    - redis-server
    - "--requirepass ${REDIS_PASSWORD}"
    - "--maxmemory 512mb"
    - "--maxmemory-policy allkeys-lru"
  volumes:
    - ./data/redis/data:/data
  networks:
    - gitlab-network
  healthcheck:
    test: ["CMD", "redis-cli", "--raw", "incr", "ping"]
    interval: 30s
    timeout: 10s
    retries: 5
  mem_limit: 2g
  cpus: 1.0
gitlab:
  image: gitlab/gitlab-ce:latest
  container_name: gitlab
  restart: always
  hostname: ${GITLAB_HOST}
  depends_on:
    gitlab-db:
      condition: service_healthy
    gitlab-cache:
      condition: service_healthy
  environment:
    GITLAB_OMNIBUS_CONFIG: |
      external_url 'http://${GITLAB_HOST}:${GITLAB_HTTP_PORT}'
      gitlab_rails['db_adapter'] = 'postgresql'
      gitlab_rails['db_encoding'] = 'unicode'
      gitlab_rails['db_host'] = 'gitlab-db'
      gitlab_rails['db_port'] = 5432
      gitlab_rails['db_username'] = '${POSTGRES_USER}'
      gitlab_rails['db_password'] = '${POSTGRES_PASSWORD}'
      gitlab_rails['db_database'] = '${POSTGRES_DB}'
      gitlab_rails['db_prepared_statements'] = false
      gitlab_rails['db_statements_limit'] = 1000
      gitlab_rails['redis_host'] = 'gitlab-cache'
      gitlab_rails['redis_port'] = 6379
      gitlab_rails['redis_password'] = '${REDIS_PASSWORD}'
      gitlab_rails['initial_root_password'] = '${GITLAB_ROOT_PASSWORD}'
      gitlab_rails['gitlab_shell_ssh_port'] = ${GITLAB_SSH_PORT}
      gitlab_rails['time_zone'] = '${TZ}'
      gitlab_rails['backup_keep_time'] = 604800
    puma['worker_processes'] = ${PUMA_WORKERS:-4}
    puma['min_threads'] = 1
    puma['max_threads'] = 4
    sidekiq['max_concurrency'] = 20
```

```

sidekiq['min_concurrency'] = 5
gitlab_rails['env'] = {
  'MALLOC_CONF' => 'dirty_decay_ms:1000,muzzy_decay_ms:1000',
  'GITLAB_RAILS_MAX_MEMORY_MB' => '4096'
}
prometheus_monitoring['enable'] = true
gitlab_rails['gitlab_default_can_create_group'] = false
logging['svlogd_size'] = 200 * 1024 * 1024
logging['svlogd_num'] = 30
logging['svlogd_timeout'] = 86400
logging['svlogd_filter'] = "gzip"
logging['svlogd_udp'] = nil
logging['svlogd_prefix'] = nil
gitlab_rails['rake_cache_clear'] = true
gitlab_rails['auto_migrate'] = false
ports:
- "${GITLAB_HTTP_PORT}:8081"
- "${GITLAB_HTTPS_PORT}:443"
- "${GITLAB_SSH_PORT}:222"
volumes:
- ./data/gitlab/config:/etc/gitlab
- ./data/gitlab/logs:/var/log/gitlab
- ./data/gitlab/data:/var/opt/gitlab
- ./data/gitlab/backups:/var/opt/gitlab/backups
networks:
- gitlab-network
healthcheck:
test: ["CMD", "/opt/gitlab/bin/gitlab-healthcheck", "--fail", "--quiet"]
interval: 120s
timeout: 60s
retries: 10
start_period: 600s
shm_size: '512m'
mem_limit: 16g
cpus: 4.0
ulimits:
nproc: 65535
nofile:
soft: 65536
hard: 65536
networks:
gitlab-network:
driver: bridge
ipam:
config:
- subnet: 172.20.0.0/16

```

Файл с определением переменных окружения .env не приводится из соображений информационной безопасности.

2. GitLab Runner: Отдельный сервис, установленный в отдельную виртуальную машину. Runner отвечает за выполнение заданий (jobs), описанных в файле .gitlab-ci.yml. В данном проекте использован runner типа shell, который для каждого задания выполняет заданный набор консольных команд.

Установка раннера на VM:

```

curl -L "https://packages.gitlab.com/install/repositories/runner/gitlab-runner/script.deb.sh" | sudo bash
sudo apt install gitlab-runner

```

3. Веб-приложение (HTML): Простое приложение с одним маршрутом, возвращающим HTML-документ. Приложение служит тестовым артефактом для сборки и тестирования в пайплайне.

Файл Index.html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>Main NA site</title>
  </head>

  <body>
    <h1>In future here will be NA main site</h1>
    <hr>
    <p>Hello from OSDA IBM MSVSIA "Eclipse"</p>
    <p>Hello from BMSTU</p>
    <hr>
    <h5>Property of Ivanov Boris Mike</h5>
  </body>
</html>
```

Принцип работы пайплайна основан на событийной модели GitLab. При пуше кода в определенные ветки (в данном случае main) автоматически запускается цепочка этапов (stages). В данном проекте настроены три этапа:

build: Сборка Docker-образа приложения по заданному Dockerfile.

```
FROM nginx:alpine
COPY src/ /usr/share/nginx/html/
EXPOSE 80
```

test: Первичная развертка и проверка созданного образа.

deploy: Запуск ранее созданного образа на постоянную работу.

Файл .gitlab-ci.yml

```
stages:
  - build
  - test
  - deploy
variables:
  SHARED_DIR: "/home/gitlab-runner/pipelines"

build:
  stage: build
  tags:
    - shell
  script:
    - docker build -t na-main-site .
    - docker save na-main-site -o $SHARED_DIR/na-main-site.tar
    - echo "Image saved to $SHARED_DIR/na-main-site.tar"
  artifacts:
    paths:
      - na-main-site.tar
test:
  stage: test
  tags:
    - shell
  script:
    - docker load -i $SHARED_DIR/na-main-site.tar
    - docker stop na-main-site 2>/dev/null || true
```

```
- docker run -d --rm --name web-test -p 8080:80 na-main-site
- sleep 3
- curl -f http://localhost:8080 || exit 1
- docker stop web-test
dependencies:
  - build
deploy:
  stage: deploy
tags:
  - shell
script:
  - docker stop na-main-site 2>/dev/null || true
  - docker rm na-main-site 2>/dev/null || true
  - docker load -i $SHARED_DIR/na-main-site.tar
  - docker run -d --name na-main-site --restart unless-stopped -p 8080:80 na-main-site
  - echo "Контейнер запущен на порту 8080"
dependencies:
  - test
```

Алгоритмическая реализация и этапы пайплайна.

Алгоритм работы CI/CD пайплайна построен на последовательном выполнении трех ключевых этапов, реализующих полный цикл сборки, тестирования и развертывания веб-приложения. Пайплайн активируется при каждом пуше кода в репозиторий GitLab и выполняется на настроенном GitLab Runner с тегом shell.

Этап 1: Build (Сборка)

На данном этапе выполняется сборка Docker-образа на основе предоставленного Dockerfile. Используется облегченный образ nginx:alpine в качестве базового. Все статические файлы из директории src/ копируются в рабочую директорию веб-сервера Nginx (/usr/share/nginx/html/). После успешной сборки образ сохраняется в виде архива (na-main-site.tar) в общей директории SHARED_DIR, что позволяет передавать артефакт между этапами пайплайна. Использование общей директории решает проблему передачи данных между джобами, выполняющимися на одном раннере.

Этап 2: Test (Тестирование)

На этапе тестирования загружается ранее сохраненный образ, после чего запускается тестовый контейнер web-test на порту 8080. Реализована проверка работоспособности веб-приложения через HTTP-запрос с использованием утилиты curl. Критическим условием успешного прохождения этапа является получение валидного HTTP-ответа (код 200) от запущенного контейнера. Для обеспечения надежности тестирования предусмотрена предварительная остановка возможных конфликтующих контейнеров и автоматическая очистка тестового контейнера после завершения проверки.

Этап 3: Deploy (Развертывание)

Финальный этап выполняет развертывание протестированного образа в продуктивную среду. Алгоритм включает:

1. Безопасную остановку и удаление предыдущей версии контейнера на-main-site
2. Загрузку актуального образа из архива
3. Запуск нового контейнера с политикой автоматического перезапуска (--restart unless-stopped)
4. Публикацию веб-приложения на порту 8080 хост-системы

Особенностью реализации является использование зависимостей (dependencies) между этапами, что гарантирует последовательное выполнение пайплайна: этап тестирования зависит от успешной сборки, а развертывание — от успешного прохождения тестов.

Экранные формы с примерами работы системы

The screenshot shows the GitLab web interface for a project named "Pipeline test-project". The interface includes a sidebar with navigation options like "Pinned", "Issues", "Merge requests", "Manage", "Plan", "Code", "Build", "Secure", "Deploy", "Operate", "Monitor", "Analyze", and "Settings". The main content area displays the project's details, including the commit history, file browser, and project information. The commit history table shows the following data:

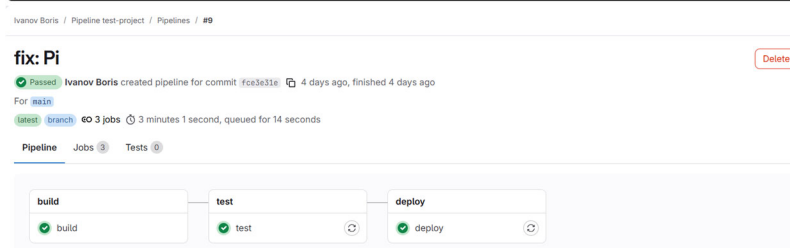
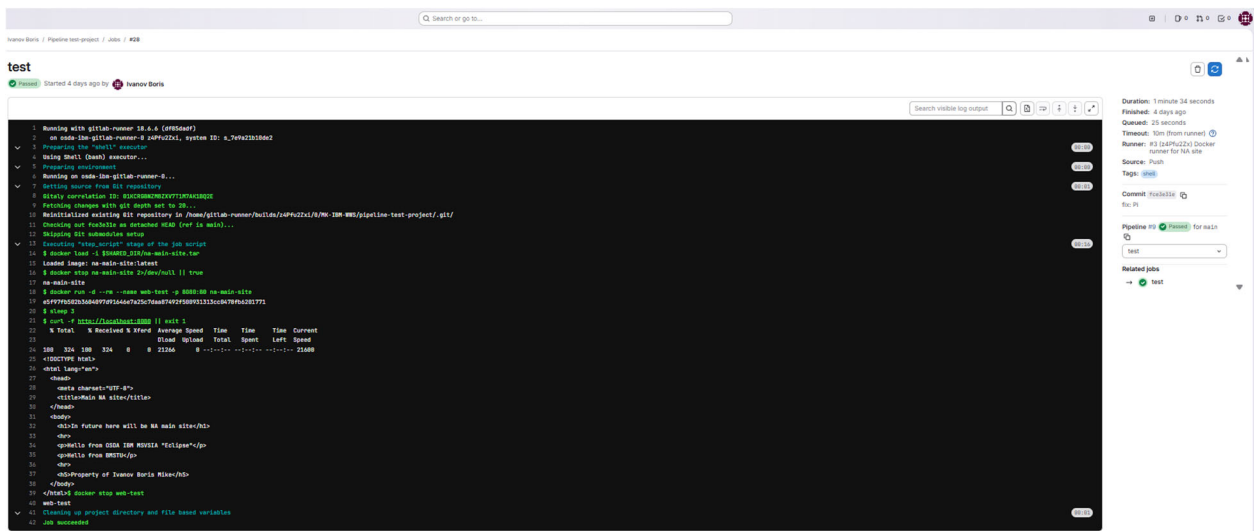
Name	Last commit	Last update
src	fix: PI	4 days ago
.gitlab-ci.yml	fix: Pipeline fix for already working site	5 days ago
Dockerfile	add: NA site pipeline base structure	6 days ago
README.md	add: NA site pipeline base structure	6 days ago

The project information sidebar on the right shows details such as "10 Commits", "1 Branch", "0 Tags", and "22 MiB Project Storage". It also includes links to "Add LICENSE", "Add CHANGELOG", "Add CONTRIBUTING", "Auto DevOps enabled", "Add Kubernetes cluster", "Add Wiki", and "Configure Integrations".

Runners

Runners are processes that pick up and execute CI/CD jobs for GitLab. [What is GitLab Runner?](#)

The screenshot shows the "Project runners" page in GitLab. It indicates that there is 1 runner assigned to the project. The runner is named "#3 (z4Pfu2Zx)" and is a "Docker runner for NA site". The runner is currently in a "shell" state. The page includes a "Create project runner" button and a "Delete runner" button.



Заключение

Результатом работы является полностью функционирующая локальная экосистема GitLab с настроенным автоматизированным пайплайном CI/CD. Проект демонстрирует практические навыки работы с контейнеризацией, оркестрацией сервисов и принципами DevOps. Полученная система позволяет автоматизировать процессы тестирования и сборки кода, повышая качество и скорость разработки. Архитектура проекта модульна и может быть расширена добавлением новых этапов в пайплайн (например, статического анализа кода, безопасности) или интеграцией с внешними системами развертывания.

